



FACULTY OF ENGINEERING / DEPARTMENT OF SOFTWARE ENGINEERING

MUĞLA SITKI KOÇMAN UNIVERSITY

Bachelor's Thesis

MycoFlow: A Bio-Inspired, Reflexive QoS System for Embedded OpenWrt Routers

Author:	Barış Can Ataklı
Supervisor:	Dr. Öğr. Üyesi Cihat Çetinkaya
Advisor:	Dr. Öğr. Üyesi Cihat Çetinkaya
Submission Date:	June 2026

Abstract

Consumer home networks must simultaneously serve latency-sensitive workloads—gaming, VoIP, and video conferencing—alongside high-throughput bulk transfers on commodity routers that operate under tight CPU, RAM, and flash constraints. CAKE Active Queue Management eliminates bufferbloat but cannot automatically classify encrypted traffic or adapt its bandwidth cap to real-time congestion conditions.

This thesis presents **MycoFlow**, a bio-inspired embedded QoS daemon for OpenWrt routers. Motivated by the information-processing strategy of fungal mycelial networks, MycoFlow runs a 1 Hz reflexive Sense→Infer→Act→Stabilize control loop entirely on-device. It classifies traffic at two complementary levels: a six-class per-device behavioral persona engine and a twelve-class per-flow service taxonomy driven by a three-signal weighted voter combining passive DNS snooping, destination-port hints, and behavioral fingerprints.

Classified flows are steered into CAKE diffserv priority tins via a CONNMARK-based DSCP pipeline. A passive BPF TCP RTT observer feeds a per-flow auto-corrector, and a feedback-driven hysteresis controller adapts the WAN bandwidth cap dynamically.

Four controlled experiments demonstrate the effectiveness of the system: (i) a **154×** lower in-tin delay for gaming versus bulk traffic under per-device DSCP marking; (ii) a **38%** reduction in gaming latency under five simultaneous mixed-persona clients compared to CAKE alone; (iii) correct per-flow service separation when a single IP concurrently runs game, VoIP, bulk, and torrent traffic; and (iv) an in-vivo DNS ablation on 1,650 stratified-random flows from the Universidad del Cauca dataset, showing passive transit-DNS lifts CAKE tin-matched accuracy from 30% to **70.4%** ($p < 10^{-63}$, McNemar), with zero flows degraded by DNS evidence.

The daemon runs at a constant 1,152 KB RSS (static CONNMARK libraries included) with median 6% CPU on a single Cortex-A53 core and writes zero bytes to NAND flash, making it suitable for deployment on consumer-grade embedded hardware.

Keywords: bufferbloat, CAKE, DSCP, eBPF, OpenWrt, QoS, traffic classification, bio-inspired networking, home network, adaptive queue management, flow-aware classification, passive TCP RTT

Özet

Tüketici ev ağlarının, CPU, RAM ve flash depolama açısından kısıtlı olan hazır yönlendiriciler üzerinde aynı anda gecikmeye duyarlı iş yüklerine (oyun, VoIP, video konferans) ve yüksek bant genişlikli toplu aktarımlara hizmet etmesi gerekmektedir. CAKE Aktif Kuyruk Yönetimi (AQM) tampon şişkinliğini ortadan kaldırmakta, ancak şifrelenmiş trafiği otomatik olarak sınıflandıramamakta ya da bant genişliği sınırını gerçek zamanlı tıkanıklığa göre ayarlayamamaktadır.

Bu tez, OpenWrt yönlendiricileri için biyolojiden ilham alan gömülü bir QoS daemon'ı olan **MycoFlow**'u sunmaktadır. Fungal misel ağlarının bilgi işleme stratejisinden esinlenen MycoFlow, cihaz üzerinde tamamen çalışan 1 Hz'lik bir Algıla→Çıkarım Yap→Harekete Geç→Sabit döngüsü yürütmektedir. Trafiği iki tamamlayıcı düzeyde sınıflandırmaktadır: pasif DNS dinleme, hedef port ipuçları ve davranışsal parmak izlerini birleştiren üç sinyalli ağırlıklı bir seçici ile yönlendirilen altı sınıflı cihaz başına davranışsal kişilik motoru ve on iki sınıflı akış başına hizmet sınıflandırması.

Dört kontrollü deney sistemin etkinliğini kanıtlamaktadır: cihaz başına DSCP işaretlemesi altında oyun ile toplu trafik için **154 kat** daha düşük kuyruk gecikmesi; beş eş zamanlı karma istemci altında yalnızca CAKE'e kıyasla oyun gecikmesinde **%38** azalma; tek bir IP'nin aynı anda oyun, VoIP, toplu ve torrent trafiği çalıştırması durumunda doğru akış başına hizmet ayırıştırması; ve pasif transit DNS dinlemenin CAKE kutu eşleştirme doğruluğunu **%30'dan %70,4'e** yükselten in-vivo DNS ablasyonu.

Anahtar Kelimeler: tampon şişkinliği, CAKE, DSCP, eBPF, OpenWrt, QoS, trafik sınıflandırma, biyolojiden ilham alan ağlar, ev ağı

Contents

Abstract	ii
Özet	iii
1. Introduction	1
1.1. Research Background	1
1.1.1. Problem Definition	2
1.2. Objectives and Contributions	2
1.3. Thesis Structure	2
2. Background	4
2.1. Bufferbloat and Active Queue Management	4
2.1.1. The Bufferbloat Problem	4
2.1.2. CoDel and fq_CoDel	4
2.1.3. CAKE: Common Applications Kept Enhanced	5
2.2. CAKE diffserv4 Mode	5
2.3. DiffServ and DSCP on Home Networks	6
2.4. Netfilter and Connection Tracking	6
2.4.1. Connection Tracking	6
2.4.2. CONNMARK	6
2.5. eBPF for Kernel-Side Telemetry	6
3. Related Work	8
3.1. AQM Evolution	8
3.2. Traffic Classification	8
3.2.1. Deep Learning Approaches	8
3.2.2. Deep Packet Inspection	8
3.3. Adaptive QoS and SQM	9
3.4. eBPF for Edge Telemetry	9
3.5. Bio-Inspired Networking	9
3.5.1. Ant Colony Optimization	9
3.5.2. Mycelial Network Research	10
3.6. Positioning	10
4. Design Rationale	11
4.1. The Mycelial Network Analogy	11

4.2.	Mycelial Principles Mapped to MycoFlow	11
4.2.1.	Hyphal Tip Sensing	11
4.2.2.	Adaptive Nutrient Redistribution	12
4.2.3.	Hysteresis and Structural Memory	13
4.2.4.	Load-Triggered Remodeling	13
4.3.	Design Principles	13
5.	System Architecture	15
5.1.	Overview	15
5.2.	Module Structure	16
5.3.	Data Flow	16
5.4.	LuCI Web Dashboard	17
5.5.	Build System and Deployment	18
6.	The Control Loop	19
6.1.	Metric Sensing and Smoothing	19
6.1.1.	RTT and Jitter	19
6.1.2.	Queue Statistics	19
6.1.3.	Flow Statistics	19
6.2.	Six-Class Per-Device Persona Engine	20
6.2.1.	Decision Tree	20
6.2.2.	Majority-Vote Stability	20
6.2.3.	Persona-to-DSCP Mapping	20
6.3.	Flow-Aware Service Classification (v3)	21
6.3.1.	Service Taxonomy	21
6.3.2.	Three-Signal Weighted Voter	22
6.3.3.	Stability Gate	22
6.4.	Congestion Detection and Bandwidth Policy	22
6.4.1.	Adaptive Thresholds	22
6.4.2.	Bandwidth Policy	23
6.5.	Action Feedback Ring	23
6.6.	Sliding Baseline	23
7.	Implementation	24
7.1.	CAKE Integration	24
7.2.	Per-Device DSCP Mangle Chain	24
7.3.	Per-Flow CONNMARK Pipeline	25
7.3.1.	Critical Kernel Module Dependency	25
7.4.	Ingress Shaping via IFB	25
7.5.	eBPF Telemetry	26
7.6.	Configuration Hierarchy	26
7.7.	Flash Safety	26
7.8.	Test Coverage	27

8. Evaluation	28
8.1. Experimental Setup	28
8.2. Phase 1: Two-Persona Validation	28
8.2.1. Setup	28
8.2.2. Gaming Latency Results	28
8.2.3. CAKE Tin Statistics	29
8.3. Phase 2: Six-Persona Mixed Workload	29
8.3.1. Setup	29
8.3.2. Latency Results	30
8.3.3. CAKE Tin Distribution	30
8.4. Phase 3: Single-IP Multi-App Flow Separation	31
8.4.1. Motivation	31
8.4.2. Setup	31
8.4.3. Results	31
8.5. Phase 4: DNS Signal Ablation and In-Vivo Classifier Accuracy	31
8.5.1. Motivation	31
8.5.2. Offline Temporal Validation	31
8.5.3. In-Vivo Validation	32
8.5.4. AF_PACKET Requirement	33
8.6. Resource Overhead	33
9. Discussion	34
9.1. Why 38% and Not More?	34
9.2. Classification Accuracy Discussion	34
9.2.1. Per-Device Heuristic Design	34
9.2.2. In-Vivo Classification Ceiling	35
9.3. Comparison with Two-Persona Results	35
9.4. Adaptive Baseline vs. Static Threshold	35
9.5. Parameter Sensitivity	36
9.6. Privacy and Security	36
9.7. Threats to Validity	36
9.8. Limitations	37
10. Conclusion	38
10.1. Summary of Contributions	38
10.2. Experimental Findings	38
10.3. Bio-Inspired Design Validation	39
10.4. Future Directions	39
10.5. Acknowledgements	40
A. Additional Materials	41
A.1. Configuration Parameters Reference	41
A.2. Unicauca Dataset Service Class Mapping	42

A.3. CAKE Tin Target Delay Summary	42
A.4. Build and Installation Instructions	42
A.4.1. Building from Source	42
A.4.2. OpenWrt Installation	43
List of Figures	44
List of Tables	45
Bibliography	46

1. Introduction

The modern household router is the single most critical network device for millions of users, yet it typically operates a fixed, administrator-configured Quality of Service (QoS) policy that was set at installation time and never updated. Meanwhile, the application mix on a home network changes continuously: a gaming session demands sub-5 ms latency, a video conference requires stable 200–400 kbps throughput with jitter below 15 ms, a streaming client needs 8–25 Mbps without packet loss, and a background BitTorrent process competes for any remaining bandwidth with hundreds of simultaneous TCP connections.

Bufferbloat [1]—the phenomenon where excess queuing inside network devices inflates round-trip time by hundreds of milliseconds—remains the dominant cause of degraded user experience on home networks. A single saturating TCP flow can fill a 1000-packet FIFO queue in under 100 ms on a 20 Mbit/s WAN link, rendering gaming and VoIP completely unusable until the flow terminates.

Active Queue Management (AQM) algorithms such as CoDel [2], fq_CoDel [3], and CAKE [4] have made significant progress against bufferbloat. CAKE in particular, which is the default AQM on OpenWrt since version 22.03, combines shaping, per-flow fair queuing, and Differentiated Services (DiffServ) priority tins in a single qdisc. However, two fundamental challenges remain unsolved for home deployments:

- **No automatic traffic classification.** CAKE’s diffserv mode relies on DSCP markings in IP headers. On home networks, most traffic arrives with CS0 (Best Effort) markings, so every flow—gaming, streaming, torrent—lands in the same tin regardless of its latency sensitivity.
- **Static WAN rate limit.** The CAKE bandwidth parameter is set once at installation. It cannot react to congestion detected through probe-based measurements or queue depth signals.

1.1. Research Background

Home network quality has become increasingly important as remote work, online gaming, and video streaming have become everyday activities. Despite advances in AQM, two layers of the problem remain unaddressed: intelligent classification of application types and dynamic adaptation of the bandwidth cap.

The biological concept of mycelial networks—the underground root-like structures of fungi—provides an unexpectedly apt analogy for the home router control problem. A fungal mycelium continuously senses its environment through hyphal tips, redistributes nutrients

adaptively using hysteresis-controlled transport, and restructures itself in response to persistent stress. These same principles—local sensing, hysteresis, and adaptive redistribution—map directly to the challenges of home QoS.

1.1.1. Problem Definition

Current home router QoS solutions face three structural limitations:

1. Traffic classification requires manual configuration by technically skilled administrators, which is unavailable to most residential users.
2. Per-device classification fails when a single host runs multiple application types simultaneously (e.g., a gaming session and a background torrent).
3. Bandwidth caps are static and do not respond to real-time congestion signals.

1.2. Objectives and Contributions

This thesis presents **MycoFlow**, a lightweight C11 daemon that addresses all three problems. Its primary objective is to enable automatic, zero-touch QoS management on consumer-grade OpenWrt routers without requiring DPI, machine learning, or external cloud infrastructure. The specific contributions of this work are:

1. A **six-class per-device persona engine** and a **twelve-class per-flow service taxonomy**, both operating without DPI or machine learning, connected by a three-signal weighted voter (passive DNS snooping, destination-port hints, behavioral fingerprints) and a CONNMARK-based DSCP pipeline.
2. A **passive BPF TCP RTT observer** (TC clsact hook) and a **feedback-driven hysteresis controller** with sliding EWMA baseline for WAN bandwidth adaptation.
3. A **practical architectural fix** for in-router DNS sniffing, demonstrating that AF_INET SOCK_RAW silently misses transit DNS from clients that bypass the local resolver.
4. An **in-vivo DNS signal ablation** ($N = 1,650$ stratified-random flows) with Wilson 95% confidence intervals and paired McNemar testing.

1.3. Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2:** Provides background on bufferbloat, AQM algorithms, CAKE, and DiffServ.
- **Chapter 3:** Surveys related work in traffic classification, adaptive QoS, and bio-inspired networking.

- **Chapter 4:** Presents the bio-inspired design rationale and mapping of mycelial principles.
- **Chapter 5:** Describes the system architecture and module structure.
- **Chapter 6:** Formalizes the reflexive control loop and its mathematical foundations.
- **Chapter 7:** Details the implementation choices, CAKE integration, eBPF telemetry, and flash safety.
- **Chapter 8:** Reports results from four controlled experiments.
- **Chapter 9:** Discusses findings, limitations, and implications.
- **Chapter 10:** Concludes and outlines future work.

2. Background

This chapter provides the technical foundations necessary to understand MycoFlow’s design. It covers the bufferbloat problem, AQM algorithms, CAKE’s architecture, and the DiffServ framework for packet prioritization.

2.1. Bufferbloat and Active Queue Management

2.1.1. The Bufferbloat Problem

When a WAN uplink is saturated, the router’s transmit queue accumulates packets faster than they can be sent. Without queue management, packets wait in FIFO order; round-trip time grows linearly with queue depth. On a 20 Mbit/s uplink with a 1000-packet queue and 1500-byte packets, the maximum queue drain time exceeds 600 ms—far beyond the 20 ms threshold for interactive applications.

This phenomenon, termed *bufferbloat* by Gettys and Nichols [1], was identified as the dominant cause of degraded user experience on home networks. A single saturating TCP flow triggers TCP’s congestion window growth, fills the router’s transmit buffer, and causes all latency-sensitive traffic (gaming, VoIP) to wait behind bulk transfer packets.

The economic incentive to add large buffers at modest cost has historically outweighed the networking community’s concern about the consequences for latency. Router manufacturers ship devices with buffers large enough to prevent buffer exhaustion at peak throughput, inadvertently maximizing queuing latency as a side effect.

2.1.2. CoDel and fq_CoDel

CoDel [2] (Controlled Delay) controls standing queue depth by measuring the minimum queuing delay experienced by packets over a sliding window. It signals congestion via ECN marks or packet drops when the minimum delay exceeds a target (typically 5 ms) for longer than an interval (100 ms). CoDel’s key insight is to measure queuing delay directly rather than queue length, making it robust to variations in packet size and link speed.

fq_CoDel [3] adds per-flow fair queuing so that a single bulk flow cannot monopolize the queue at the expense of interactive flows. By hash-routing packets into sparse flow buckets, each with independent CoDel management, fq_CoDel ensures that a BitTorrent swarm with 200 TCP connections does not inflict its queuing delay on a single VoIP packet.

2.1.3. CAKE: Common Applications Kept Enhanced

CAKE [4] subsumes both CoDel and fq_CoDel and adds three layers of priority management:

- **Per-tin bandwidth allocation:** Traffic is separated into priority tiers (tins), each with guaranteed minimum bandwidth and an independent AQM target.
- **Per-flow isolation within each tin:** Within each tin, flows are hash-routed to minimize head-of-line blocking.
- **Per-host round-robin within each flow set:** Ensures fairness between hosts sharing the same tin.

CAKE is the default AQM on OpenWrt since version 22.03 and is configured via the `tc` command. Its `diffserv4` mode partitions traffic into four tins based on the IP Precedence field (upper three bits of DSCP): Voice (Precedence 4–7), Video (Precedence 2–3), Best Effort (Precedence 0), and Bulk (Precedence 1).

2.2. CAKE `diffserv4` Mode

CAKE's `diffserv4` mode partitions traffic into four tins based on the IP Precedence field (upper three bits of DSCP):

- **Voice tin** (Precedence 4–7): Served first; designed for VoIP and gaming traffic.
- **Video tin** (Precedence 2–3): High-priority video streaming traffic.
- **Best Effort tin** (Precedence 0): Default for all unmarked traffic.
- **Bulk tin** (Precedence 1): Background transfers that should not affect interactive applications.

Each tin has an independent minimum guaranteed rate and an AQM target delay. Under CAKE `diffserv4`, a packet with DSCP CS4 (Precedence 4) enters the Voice tin and is served before any Best Effort or Bulk packet.

The DSCP-to-tin mapping used by MycoFlow is:

- EF → Voice (VoIP)
- CS4 → Voice (gaming)
- CS3 → Video (video call)
- CS2 → Video (streaming)
- CS1 → Bulk (torrent, bulk transfer)

MycoFlow exploits this mechanism by correctly marking per-device and per-flow traffic, shifting it from the default Best Effort tin into the appropriate priority tier without any modification to CAKE itself.

2.3. DiffServ and DSCP on Home Networks

The Differentiated Services architecture [5] defines per-hop behaviors based on the DSCP field of the IP header. DSCP occupies the upper six bits of the former IPv4 Type of Service (ToS) byte and the IPv6 Traffic Class field.

Edge routers at ISP boundaries are expected to re-mark traffic entering their network, but most ISP CPE (Customer Premises Equipment) devices either ignore DSCP or clear it entirely. Home devices typically generate unmarked traffic (CS0, Best Effort), and no automatic mechanism exists to reclassify it at the home gateway.

This creates the fundamental gap that MycoFlow addresses: the CAKE qdisc is present and fully functional on OpenWrt routers, but without correctly classified DSCP markings, all traffic lands in the Best Effort tin regardless of its latency requirements.

2.4. Netfilter and Connection Tracking

Linux's Netfilter subsystem provides packet filtering, NAT, and connection tracking capabilities used by OpenWrt. The `iptables` and `nftables` frontends configure Netfilter rules; the `nf_conntrack` kernel module tracks connection state.

2.4.1. Connection Tracking

The `conntrack` table (`/proc/net/nf_conntrack`) maintains state for each active network connection as a 5-tuple (source IP, destination IP, source port, destination port, protocol). For each entry, it records byte and packet counts in both directions, enabling per-flow statistics without kernel modifications.

2.4.2. CONNMARK

CONNMARK is a Netfilter mechanism that stores a 32-bit mark on a `conntrack` entry. Unlike per-packet marks (MARK), which are ephemeral, a CONNMARK persists for the lifetime of the connection. An `iptables` rule can save a packet's mark to the `conntrack` entry (CONNMARK -save-mark) or restore the `conntrack` mark to the packet (CONNMARK -restore-mark).

This mechanism is central to MycoFlow's per-flow DSCP pipeline: once a flow is classified, a CONNMARK stores the service class, and a POSTROUTING rule restores it to DSCP on every subsequent packet without re-running the classifier.

2.5. eBPF for Kernel-Side Telemetry

Extended Berkeley Packet Filter (eBPF) allows user-defined programs to run in the Linux kernel in a verified, sandboxed environment. For networking, eBPF programs can be attached to TC (Traffic Control) hooks, XDP (eXpress Data Path), and socket-level filters.

MycoFlow uses a TC clsact hook to sample passive TCP RTT from TCP timestamp options (TSval/TSecr pairs). The BPF program maintains a per-flow smoothed RTT in a BPF hash map that the userspace daemon polls each cycle, enabling RTT-driven flow demotion without active probing overhead.

The eBPF integration is optional: when `libbpf` is unavailable or loading fails, the daemon falls back gracefully to netlink-only statistics.

3. Related Work

This chapter surveys existing approaches to bufferbloat mitigation, traffic classification, adaptive QoS, and bio-inspired networking, positioning MycoFlow relative to each area.

3.1. AQM Evolution

Proportional Integral controller Enhanced (PIE) [6] achieves CoDel-equivalent performance with lower computational cost by modeling the queue delay as a control system. FQ-PIE combines PIE with per-flow queuing. CAKE [4] remains the most comprehensive open-source AQM for home gateways, integrating shaping, fair queuing, and DiffServ. None of these systems perform automatic per-device DSCP classification; they all require pre-classified traffic to deliver tier-specific service.

3.2. Traffic Classification

3.2.1. Deep Learning Approaches

GGFAST [7] applies payload-agnostic deep learning to encrypted traffic fingerprinting at line rate; it achieves high accuracy but requires offline model training infrastructure and cloud-side catalog updates, making continuous adaptation incompatible with embedded routers that lack the compute and storage for model inference.

Lotfollahi et al. [8] use deep learning on raw packet features, achieving high accuracy on encrypted traffic. Again, the resource requirements (GPU inference, gigabytes of training data) preclude embedded deployment on a 256 MB router.

3.2.2. Deep Packet Inspection

nDPI [9] provides open-source DPI classification but requires payload parsing, raising privacy concerns and imposing significant CPU cost on constrained hardware. While an OpenWrt package exists for nDPI, its per-packet payload inspection overhead is incompatible with constrained single-core SoCs running concurrent Wi-Fi, NAT, and AQM tasks.

MycoFlow deliberately avoids both ML and DPI: its heuristic decision tree runs in under $1\ \mu\text{s}$ per device per cycle on a Cortex-A53 core, requiring no model files and no payload access.

3.3. Adaptive QoS and SQM

SDN-based per-application QoS systems achieve fine-grained traffic differentiation but require a network controller external to the CPE, adding deployment complexity incompatible with consumer home settings where no IT staff are available.

OpenWrt `qosify` provides static DSCP mapping via `nftables` rules with fixed port-to-class tables, but no runtime sensing or adaptation. It offers a partial solution for technically skilled users who manually maintain port tables, but cannot respond to changes in traffic patterns or network conditions.

SQM (Smart Queue Management, `luci-app-sqm`) is the canonical OpenWrt QoS solution [4]: it configures CAKE or fq_CoDel with user-specified bandwidth limits and applies AQM. However, SQM has three structural limitations that MycoFlow addresses:

1. **Static bandwidth cap:** SQM requires the operator to manually configure the WAN bandwidth cap once at installation; it does not adjust the cap in response to observed congestion.
2. **No automatic classification:** If a user wants gaming traffic to enter CAKE's Voice tin, they must manually assign DSCP marks on each game client—an operation requiring per-device network configuration knowledge.
3. **No per-flow distinction:** SQM cannot distinguish between multiple applications running simultaneously on the same host (e.g., a gaming session and a background torrent), because it operates at the qdisc level with no per-flow state.

MycoFlow is designed as a *zero-touch complement* to SQM: it deploys CAKE identically but adds autonomous sensing, classification, and actuation so that no manual DSCP tagging or bandwidth tuning is ever required.

3.4. eBPF for Edge Telemetry

Magnani et al. [10] demonstrate adaptive eBPF-based monitoring pipelines on edge routers, showing that kernel-side counters can expose per-flow statistics with negligible overhead. Their work validates the feasibility of eBPF for in-router telemetry and informs MycoFlow's passive TCP RTT observer.

MycoFlow uses eBPF for passive TCP RTT sampling via a TC `clsact` hook but primarily relies on `/proc/net/nf_conntrack` for flow statistics, as eBPF kernel headers are not available on all OpenWrt builds.

3.5. Bio-Inspired Networking

3.5.1. Ant Colony Optimization

Ant Colony Optimization [11] has been applied to routing in multi-hop networks. The core idea—that distributed agents following simple local rules can produce globally optimal

paths—maps to MycoFlow’s per-device decision making. However, ACO requires agent communication infrastructure absent in home routers; MycoFlow achieves similar local-sensing behavior with purely passive observation.

3.5.2. Mycelial Network Research

Adamatzky and colleagues [12] characterize mycelial networks as fault-tolerant distributed logic gates, demonstrating that biological neural-like computation can emerge from simple local chemical signaling.

Fricker et al. [13] model nutrient transport in mycelial networks as an adaptive flow optimization problem, showing that transport channels in mycelium operate with hysteresis: a sustained signal above a threshold triggers remodeling, preventing oscillation.

Fukasawa et al. [14] analyze how foraging heuristics in mycelium maximize resource extraction under uncertainty, finding that hysteresis-controlled redistribution outperforms reactive strategies in dynamic environments.

MycoFlow is the first system to apply mycelial foraging and transport principles specifically to the QoS control plane of an embedded router.

3.6. Positioning

Table 3.1 summarizes the comparison of related approaches against MycoFlow. MycoFlow occupies a unique design point: it runs entirely on the CPE, requires no cloud or SDN controller, uses no ML or DPI, and automatically classifies and steers per-device traffic with runtime adaptation.

Table 3.1.: Comparison of Related QoS Approaches. Emb.=runs fully on CPE; Auto=automatic classification; No ML=no ML model required; No DPI=no payload inspection; Adapt=runtime adaptation.

System	Emb.	Auto	No ML	No DPI	Adapt
SQM/CAKE	✓	×	✓	✓	×
qosify	✓	×	✓	✓	×
nDPI	✓ [†]	✓	✓	×	×
GGFAST	×	✓	×	✓	×
SDN-QoS	×	✓	✓	✓	✓
MycoFlow	✓	✓	✓	✓	✓

[†]nDPI imposes significant CPU cost on constrained hardware.

4. Design Rationale

This chapter presents the biological motivation for MycoFlow’s architecture and maps each mycelial mechanism to a concrete engineering decision. We do not claim that our mechanisms are algorithmically derived from biology; rather, the analogy clarifies why particular engineering choices are well-suited to the embedded QoS domain.

4.1. The Mycelial Network Analogy

MycoFlow’s architecture is motivated by an analogy with fungal mycelial networks [13, 14], which face a structurally similar resource allocation problem: a single nutrient source must be distributed among competing consumers with heterogeneous requirements, under dynamic and partially observable conditions.

A fungal mycelium consists of a network of hyphae (thread-like filaments) that grow from a central colony into the surrounding environment. As hyphae explore the soil, they sense chemical gradients from nutrient sources and redirect growth toward productive regions. When a rich nutrient source is found, transport channels widen; when a source is depleted, channels narrow or retract. This adaptive behavior emerges from purely local chemical signaling without any central controller.

The home router faces an analogous problem. The WAN link is a single shared resource. Multiple clients—gaming consoles, laptops, streaming devices, phones—compete for bandwidth with heterogeneous requirements. The optimal allocation changes as application patterns shift throughout the day. No central authority classifies traffic; all classification must be inferred from observable signals at the router.

4.2. Mycelial Principles Mapped to MycoFlow

Table 4.1 maps the four key mycelial mechanisms to their MycoFlow engineering equivalents.

4.2.1. Hyphal Tip Sensing

In mycelium, hyphal tips are the primary sensory organs: they detect chemical concentration gradients and relay signals back to the transport network. MycoFlow implements its sensing layer through multiple complementary probes:

- **ICMP multi-ping:** Three echo requests per cycle to a configurable probe host measure mean RTT and standard deviation (jitter). This provides a direct measurement of user-perceived latency.

Table 4.1.: Mycelial Principles Mapped to MycoFlow Mechanisms

Biological Mechanism	MycoFlow Engineering Equivalent
Hyphal tip sensing	ICMP multi-ping probes; Netlink qdisc statistics; conntrack flow statistics; passive DNS snooping; BPF passive TCP RTT observer
Adaptive nutrient redistribution	Per-device and per-flow DSCP marking into CAKE priority tins (via iptables mangle chain and CONNMARK restore)
Hysteresis & structural memory	Majority-vote window (k -of- m over sliding history); two-tick flow stability gate; action cooldown timers
Load-triggered re-modeling	Sliding EWMA baseline; step adaptation via action feedback ring; per-flow RTT demote/re-promote ladder

- **Netlink qdisc statistics:** Raw NETLINK_ROUTE socket queries retrieve CAKE’s backlog, drops, and overlimits per cycle. Non-zero backlog is a kernel-side bufferbloat indicator, faster and more reliable than RTT alone.
- **conntrack flow statistics:** `/proc/net/nf_conntrack` provides per-flow byte counts in both directions, enabling device-level behavioral profiling without packet capture.
- **Passive DNS snooping:** An AF_PACKET socket on the LAN-facing interface captures DNS responses, building a hostname-to-IP cache that enriches flow classification.
- **BPF passive TCP RTT:** A TC clsact eBPF program samples TCP timestamp options, maintaining per-flow smoothed RTT in a BPF map.

The diversity of sensing channels provides redundancy analogous to mycelial sensing: if one signal is unavailable (e.g., eBPF not supported on a kernel build), the system continues operating with reduced fidelity rather than failing.

4.2.2. Adaptive Nutrient Redistribution

Mycelial transport channels redistribute nutrients toward productive consumers and away from depleted regions. MycoFlow’s equivalent is DSCP-based traffic steering: once a device or flow is classified, DSCP marks redirect it into the appropriate CAKE tin, ensuring that gaming and VoIP packets receive Voice tin priority while bulk transfers are confined to the Bulk tin.

The key engineering insight is that DSCP marks are persistent once applied—a packet’s DSCP travels with it to the network edge. For per-flow CONNMARK marks, the persistence is at the conntrack level: once marked, all future packets of a flow are DSCP-restored by the iptables POSTROUTING rule without re-running the classifier.

4.2.3. Hysteresis and Structural Memory

Mycelial networks do not oscillate between resource allocation states because transport channels have inertia: they require a sustained signal above a threshold before remodeling occurs. This hysteresis prevents wasted energy on transient fluctuations.

MycoFlow implements three hysteresis mechanisms:

1. **k -of- m majority vote** (default $k = 3$, $m = 5$ cycles): A device's persona changes only when the same class wins in at least 3 of the 5 most recent cycles. This prevents rapid oscillation from momentary traffic spikes.
2. **Two-tick flow stability gate**: A flow-level classification decision is committed only after it has been observed in two consecutive cycles. Short-lived DNS queries and TCP handshakes do not trigger CONNMARK writes.
3. **Action cooldown timers**: WAN bandwidth adjustments are gated by a minimum interval ($\tau = 3$ s) and a maximum action rate ($\rho = 0.5$ Hz), preventing control loop oscillation.

4.2.4. Load-Triggered Remodeling

When mycelium encounters sustained resource stress, it undergoes structural remodeling: existing pathways widen or narrow, and new growth is redirected. MycoFlow's equivalent is the adaptive baseline and feedback ring:

- **Sliding EWMA baseline**: The idle RTT and jitter baselines drift toward current conditions at a rate of 1% per minute, preventing the controller from becoming permanently anchored to favorable boot-time conditions.
- **Action feedback ring**: A circular buffer records the effects of actuation. If repeated adjustments fail to reduce RTT, the step size is halved, preventing the controller from thrashing.
- **Per-flow RTT demote/re-promote ladder**: Individual flows that persistently exceed their service RTT target are demoted to a lower-priority class, then re-promoted when conditions recover.

4.3. Design Principles

The mycelial analogy motivates six design principles that constrain every component of MycoFlow:

1. **Local decision-making**: All classification and control decisions derive from metrics collected on the router itself. No cloud service, SDN controller, or external database is required.

2. **Graceful degradation:** When a sensing channel is unavailable (e.g., eBPF not loaded, DNS cache cold), the system operates with reduced precision rather than failing.
3. **Flash safety:** No state is written to NAND flash during normal operation. All state is RAM-resident and resets cleanly on reboot.
4. **Hysteresis over reactivity:** Stability is preferred over rapid response. A 1-second reaction latency is acceptable in exchange for elimination of oscillation.
5. **Zero DPI, zero ML:** All classification runs from header-visible metadata (IP addresses, ports, packet counts, sizes) and DNS hostnames. No payload bytes are read.
6. **Composability:** MycoFlow augments CAKE; it does not replace it. Removing MycoFlow restores the system to vanilla CAKE behavior.

5. System Architecture

This chapter describes the overall architecture of MycoFlow, its component modules, and how they interact to form the reflexive control loop.

5.1. Overview

MycoFlow consists of seven components deployed on the OpenWrt router:

1. **mycoflowd** — the main control daemon, a single static binary compiled for `aarch64-openwrt-linux-musl`.
2. A **CAKE qdisc** on the WAN egress interface, configured with `diffserv4`.
3. An optional **CAKE qdisc on an IFB device** for ingress shaping.
4. An **iptables mangle chain** (`mycoflow_dscp`) for per-device DSCP marking.
5. An **iptables mangle POSTROUTING chain** that restores per-flow CONNMARKs to DSCP.
6. An optional **BPF TC clsact program** that samples passive TCP RTT on the WAN interface.
7. A **LuCI web dashboard** served from `/tmp/myco_state.json`.

The daemon runs at a configurable rate (default 1 Hz) and executes four phases per cycle, as shown in Figure 5.1:

1. **Sense:** Collect RTT and jitter via ICMP probes, interface byte/packet counters via `/proc/net/dev`, qdisc statistics via `NETLINK_ROUTE`, flow statistics from `/proc/net/nf_contrack`, passive DNS responses from the local resolver, and per-flow smoothed TCP RTT from the BPF map.
2. **Infer:** Apply EWMA smoothing to raw metrics; run the six-class per-device persona engine and update the system-wide persona via majority vote; run the three-signal weighted voter over each active 5-tuple to assign a flow-level service class.
3. **Act:** If the persona changed, update CAKE AQM target latency and apply per-device DSCP mangle rules. For each newly stable flow, push a CONNMARK so the iptables mangle POSTROUTING chain restores its DSCP on egress. If a flow's RTT exceeds its service target for two ticks, demote it one tier.

4. **Stabilize:** Record actuation outcomes in the feedback ring; evaluate ring for step-size adaptation; update the sliding baseline.

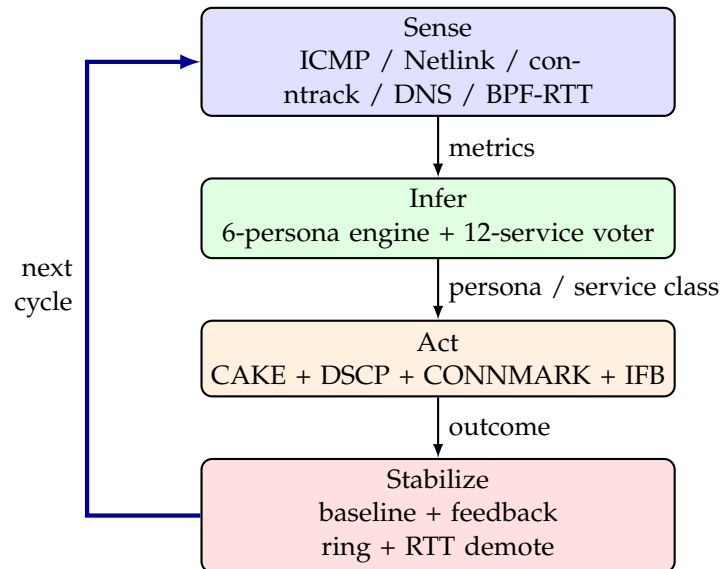


Figure 5.1.: MycoFlow reflexive control loop. Each phase executes once per second.

5.2. Module Structure

The daemon comprises 20 C11 source modules (Table 5.1) totalling approximately 4,500 lines of code. The modules are grouped into three layers:

- **Core 1 Hz reflexive loop:** Sensing, EWMA, congestion detection, actuation, configuration, ubus IPC, and logging.
- **Per-device persona layer:** Six-class decision tree, per-device state table, majority-vote history window.
- **Flow-aware v3 layer:** Twelve-class service taxonomy, three-signal classifier, passive DNS cache, CONNMARK push engine, passive RTT observer.

5.3. Data Flow

Figure 5.2 illustrates the complete data flow from packet arrival to CAKE tin assignment.

Table 5.1.: MycoFlow Source Modules and Responsibilities

Module	Responsibility
<i>Core reflexive loop</i>	
myco_sense	ICMP multi-ping, /proc/net/dev, CPU load
myco_control	EWMA, adaptive thresholds, feedback ring
myco_act	tc/iptables actuation, IFB setup
myco_ewma	Generic EWMA filter library
myco_netlink	NETLINK_ROUTE qdisc statistics
myco_ebpf	BPF map reading, TC hook management
myco_config	UCI + environment + compiled defaults
myco_ubus	ubus IPC + /tmp/myco_state.json export
myco_log	Structured stdout logging
<i>Per-device persona layer</i>	
myco_persona	6-class decision tree, majority-vote window
myco_flow	LRU conntrack table, elephant flow detection
myco_device	Per-device persona table, DSCP rule lifecycle
<i>Flow-aware v3 layer</i>	
myco_service	12-class service taxonomy + RTT demote ladder
myco_hint	Destination-port hint table
myco_dns	Passive DNS snooping cache (hostname→IP)
myco_profile	Per-flow behavioral fingerprint
myco_classifier	3-signal weighted voter + stability gate
myco_mark	libnetfilter_conntrack CT-mark push
myco_mangle	iptables mangle POSTROUTING DSCP restore
myco_rtt	libbpf TC-clasact passive TCP RTT observer

5.4. LuCI Web Dashboard

The daemon exports a JSON state snapshot to /tmp/myco_state.json (tmpfs, not NAND flash) every cycle via the ubus IPC mechanism. The LuCI dashboard package reads this file and renders:

- Per-device persona table showing IP address, persona class, DSCP mark, and CAKE tin.
- Per-flow service table showing 5-tuple, service class, current DSCP, RTT, and demotion status.
- Bandwidth cap history graph showing the adaptive adjustment timeline.
- CAKE tin statistics (packets, bytes, average delay) per tin.

The dashboard is entirely read-only; all control decisions are made by the daemon. No runtime configuration changes are made through the LuCI interface during normal operation.

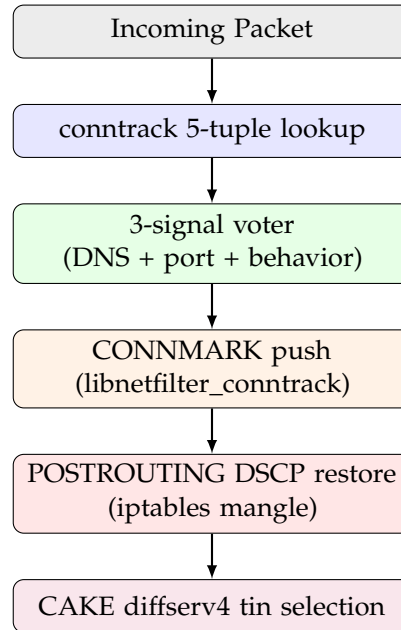


Figure 5.2.: Per-flow data path from packet arrival to CAKE tin assignment.

5.5. Build System and Deployment

MycoFlow is built using the OpenWrt SDK with a CMake build system. The binary is statically linked against musl libc, libnetfilter_conntrack, libnfnetlink, libmnl, and libbbpf (optional). The resulting mycoflowd binary is approximately 744 KB on aarch64.

The daemon is managed by OpenWrt's procd process supervisor, which provides automatic restart on crash and log collection to the RAM-resident logd ring buffer. Configuration is stored in OpenWrt UCI (/etc/config/mycoflow) and can be hot-reloaded by sending SIGHUP to the daemon.

6. The Control Loop

This chapter formalizes the mathematical foundations of MycoFlow’s reflexive control loop, covering metric sensing and smoothing, the six-class persona classification engine, the twelve-class flow-level service taxonomy, and the congestion detection and bandwidth adaptation policy.

6.1. Metric Sensing and Smoothing

6.1.1. RTT and Jitter

Every cycle, `myco_sense` issues three ICMP echo requests to a configurable probe host (default 1.1.1.1), measuring mean RTT and standard deviation (jitter) from the replies. Packet loss is computed as:

$$\ell_t = \frac{N_{\text{sent}} - N_{\text{received}}}{N_{\text{sent}}} \quad (6.1)$$

Raw measurements are smoothed by an Exponentially Weighted Moving Average (EWMA) with parameter α :

$$\hat{r}_t = \alpha r_t + (1 - \alpha) \hat{r}_{t-1}, \quad \hat{j}_t = \alpha j_t + (1 - \alpha) \hat{j}_{t-1}. \quad (6.2)$$

The default $\alpha = 0.30$ provides a time constant of approximately 3 cycles (3 s at 1 Hz), balancing responsiveness and noise rejection.

6.1.2. Queue Statistics

A raw `NETLINK_ROUTE` socket (`RTM_GETQDISC`) retrieves the CAKE qdisc’s backlog (bytes currently queued), drops, and overlimits each cycle. A non-zero backlog is a direct kernel-side bufferbloat indicator, faster and more reliable than the RTT probe alone because it detects congestion before it propagates to the probe host RTT.

6.1.3. Flow Statistics

`myco_flow` parses `/proc/net/nf_conntrack` into a userspace LRU hash table (256 entries, FNV-1a hash). For each entry, both forward (`src→dst`) and reverse (`dst→src`) byte counts are extracted; the reverse direction carries download volume, which is essential for distinguishing STREAMING (high download, low upload) from BULK (high download, high upload). Stale entries (no activity for 60 s) are evicted each cycle.

An *elephant flow* is defined as a single flow carrying more than 60% of a device’s total bytes in the measurement window.

6.2. Six-Class Per-Device Persona Engine

`myco_device` maintains a per-device record keyed by source IP. Each cycle, flows from the `conntrack` table are aggregated per source IP, and the device's behavioral profile is updated: total active flows, total bytes, reverse bytes, average packet size, and elephant flow flag.

6.2.1. Decision Tree

The six personas are inferred via the priority-ordered decision tree shown in Table 6.1. The conditions are evaluated in order; the first matching rule wins.

Table 6.1.: Persona Inference Decision Table (first matching rule wins)

Condition (priority order)	Persona
$F > 30$	TORRENT
E and $B_{rx}/B < 0.25$	STREAMING
E	BULK
$\bar{s} < 120$ B and $\dot{B} < 200$ kbps	VOIP
$\bar{s} < 350$ B and $F < 8$	GAMING
$200 \text{ kbps} \leq \dot{B} \leq 8 \text{ Mbps}$	VIDEO
otherwise	UNKNOWN

F : active flows; E : elephant flag; B_{rx} : reverse bytes; B : total bytes;
 $\bar{s}$: avg packet size; $\dot{B}$: device throughput (bytes/s).

6.2.2. Majority-Vote Stability

A three-of-five majority vote over a sliding history window prevents rapid class oscillation. Let $h_{t-4}, \dots, h_t$ be the raw persona inferences from the five most recent cycles. The current persona is updated only when a single class c^* satisfies:

$$\sum_{i=t-4}^t \mathbf{1}[h_i = c^*] \geq k, \quad k = 3 \quad (6.3)$$

If no class satisfies this condition, the previous persona is retained.

6.2.3. Persona-to-DSCP Mapping

Each persona maps to a DSCP value enforced by an iptables mangle rule in the `mycoflow_dscp` chain:

When a device's persona changes, the old mangle rule is removed and a new rule is inserted atomically, ensuring no packets are classified under an intermediate state.

Table 6.2.: Persona-to-DSCP-to-CAKE-Tin Mapping

Persona	DSCP	Prec.	CAKE Tin
VOIP	EF	6	Voice
GAMING	CS4	4	Voice
VIDEO	CS3	3	Video
STREAMING	CS2	2	Video
BULK	CS1	1	Bulk
TORRENT	CS1	1	Bulk
UNKNOWN	CS0	0	Best Effort

6.3. Flow-Aware Service Classification (v3)

The per-device persona layer delivers correct behavior when each LAN host runs a single dominant application, but collapses when one source IP simultaneously carries a video call, a bulk download, and a torrent swarm. MycoFlow v3 adds a second classification axis operating on individual conntrack 5-tuples.

6.3.1. Service Taxonomy

Twelve service classes cover the traffic types commonly seen on home links (Table 6.3). Each class carries a target RTT, a default DSCP mark, and a demotion successor for the RTT auto-corrector.

Table 6.3.: Flow-Level Service Taxonomy

Service Class	DSCP	Target RTT	Demotes to
game_rt	EF	20 ms	web_interactive
voip_call	EF	30 ms	web_interactive
video_conf	CS4	50 ms	web_interactive
video_live	CS3	100 ms	bulk_dl
video_vod	CS2	150 ms	bulk_dl
web_interactive	CS2	80 ms	bulk_dl
game_launcher	CS1	200 ms	bulk_dl
file_sync	CS1	200 ms	bulk_dl
bulk_dl	CS1	250 ms	torrent
torrent	CS1	400 ms	torrent
system	CS0	—	unknown
unknown	CS0	—	unknown

6.3.2. Three-Signal Weighted Voter

For every active conntrack entry, `myco_classifier` produces a score for each candidate service class by summing three independently normalized signals:

$$S_c(f) = w_{\text{dns}} D_c(f) + w_{\text{port}} P_c(f) + w_{\text{behav}} B_c(f), \quad (6.4)$$

with weights $(w_{\text{dns}}, w_{\text{port}}, w_{\text{behav}}) = (0.6, 0.3, 0.1)$ and an acceptance threshold $\theta = 0.3$.

The three signal components are:

- $D_c(f)$ — **DNS signal**: A passive DNS snooping cache records, for each resolved name, the set of service classes its suffix pattern matches (e.g., `discord.gg` → `VOIP_CALL`). $D_c(f) = 1$ if the flow's destination IP matches a cached hostname associated with class c , otherwise 0.
- $P_c(f)$ — **Port hint signal**: A destination-port hint table maps well-known ports to service classes (e.g., UDP/3478 STUN → `VOIP_CALL`, UDP/27015 Source-engine → `GAME_RT`). $P_c(f) \in [0, 1]$ based on port match quality.
- $B_c(f)$ — **Behavioral fingerprint**: `myco_profile` computes mean packet size, packets-per-second, inter-arrival jitter, and burstiness. $B_c(f) \in [0, 1]$ based on how well the flow's behavioral fingerprint matches the expected profile for class c .

The class with the highest score above θ wins; otherwise the flow remains `UNKNOWN` and is re-evaluated next cycle.

6.3.3. Stability Gate

A classification decision is committed only after it has been observed in two consecutive ticks (≈ 1 s at 1 Hz). This suppresses oscillation from short-lived flows (DNS queries, TCP handshakes) that never accumulate sufficient signal.

6.4. Congestion Detection and Bandwidth Policy

6.4.1. Adaptive Thresholds

Let $\mathcal{L} = \{\text{VOIP, GAMING, VIDEO}\}$ (latency-sensitive) and $\mathcal{B} = \{\text{BULK, STREAMING, TORRENT}\}$ (bulk-tolerant). Congestion is declared when any of the following conditions hold:

$$\hat{r}_t > r_{\text{base}} \cdot (1 + \gamma) \quad (6.5)$$

$$\hat{j}_t > j_{\text{base}} \cdot (1 + \gamma) \quad (6.6)$$

$$\text{backlog}_t > 0 \quad (6.7)$$

$$\ell_t > 0.02 \quad (6.8)$$

where $r_{\text{base}}, j_{\text{base}}$ are the sliding baselines, $\gamma = 0.30$ is the margin factor (tunable), and ℓ_t is the probe packet loss fraction. The thresholds are clamped to prevent false triggers on near-zero baselines: $\theta_r \in [8, 60]$ ms, $\theta_j \in [4, 30]$ ms.

6.4.2. Bandwidth Policy

When congestion is detected or cleared, the WAN bandwidth cap is adjusted:

$$u_{t+1} = \begin{cases} u_t - \lfloor \Delta/2 \rfloor & \text{cong., } p \in \mathcal{L} \\ u_t - \Delta & \text{cong., } p \in \mathcal{B} \\ u_t + \Delta & \text{clear, } p \in \{\text{VOIP, GAMING}\} \\ u_t & \text{otherwise,} \end{cases} \quad (6.9)$$

where $\Delta \equiv \Delta_{\text{step}}$ (default 2,000 kbit/s) and $u_t \in [u_{\min}, u_{\max}]$. Latency-sensitive personas receive a gentler reduction to preserve forward progress; bulk-tolerant personas are throttled more aggressively. Actions are gated by a minimum interval (cooldown $\tau = 3$ s) and a maximum action rate ρ (default 0.5 Hz).

6.5. Action Feedback Ring

A circular buffer of $N = 8$ slots records the effect of each actuation. Each slot stores: timestamp t_k , bandwidth before u_k^- , bandwidth after u_k^+ , and RTT before r_k^- . Three cycles after the actuation, the RTT after r_k^+ is filled in from the current measurement.

The ring is evaluated whenever a new record is filled: if ≥ 4 filled records exist and more than half satisfy $r_k^+ \geq r_k^- - 2.0$ ms (no RTT improvement), the step size is halved (minimum 500 kbit/s) and the `step_adapted` flag is set. This prevents the controller from repeatedly issuing adjustments that fail to improve latency.

6.6. Sliding Baseline

The idle baseline ($r_{\text{base}}, j_{\text{base}}$) is established at startup using a blocking N_b -sample average (default $N_b = 5$). During normal operation, the baseline drifts toward current conditions every T_b cycles (default $T_b = 60$) using a slow EWMA:

$$r_{\text{base}}^{(t+T_b)} = (1 - \beta) r_{\text{base}}^{(t)} + \beta \hat{r}_t, \quad \beta = 0.01. \quad (6.10)$$

The same update applies to j_{base} . The drift rate of 1% per minute prevents the baseline from permanently anchoring to favorable conditions observed at boot, while being slow enough to ignore short-lived traffic bursts.

7. Implementation

This chapter describes the concrete implementation choices that realize MycoFlow’s architecture on OpenWrt embedded hardware, including CAKE integration, ingress shaping, eBPF telemetry, configuration management, flash safety, and test coverage.

7.1. CAKE Integration

CAKE is configured with `diffserv4` and a `rtt` parameter that sets the AQM target delay. MycoFlow sets the CAKE `rtt` value per persona on the egress (WAN) interface using `tc qdisc change`:

- VOIP / GAMING → 50 ms AQM target
- VIDEO → 75 ms AQM target
- STREAMING → 150 ms AQM target
- BULK / TORRENT → 200 ms AQM target

The actuation uses `change` first and falls back to `replace` if the `qdisc` does not yet exist, ensuring idempotent operation. The CAKE `rtt` parameter affects the internal AQM target delay as $\text{target} \approx \text{rtt}/20$, so setting `rtt=50ms` yields a 2.5 ms AQM target in the GAMING persona.

7.2. Per-Device DSCP Mangle Chain

Per-device DSCP marking is implemented via an `iptables` mangle chain inserted before `POSTROUTING`:

1. At startup, MycoFlow creates the `mycoflow_dscp` chain and adds a jump rule in the `PREROUTING` table.
2. When a device’s persona is assigned, an `iptables` rule is added: `-s <device_ip> -j DSCP -set-dscp-class <class>`.
3. When the persona changes, the old rule is removed and a new rule is inserted atomically via a two-step `iptables -D / iptables -I` sequence.
4. At shutdown, the chain is flushed and the jump rule is removed.

This approach ensures that DSCP marking persists even if the daemon is briefly restarted, as the iptables rules survive daemon restarts.

7.3. Per-Flow CONNMARK Pipeline

The per-flow classification pipeline requires two iptables mangle rules that work in concert:

1. **CONNMARK restore rule** (PREROUTING): Restores the stored conntrack mark to the packet mark at ingress: `-m conntrack -ctstate ESTABLISHED,RELATED -j CONNMARK -restore-mark`.
2. **DSCP restore rule** (POSTROUTING): Sets the DSCP field from the packet mark: `-m mark ! -mark 0 -j DSCP -set-dscp-from-mark`.

When the daemon classifies a flow as a specific service class, it calls `nfct_query(NFCT_Q_UPDATE)` via `libnetfilter_conntrack` to set the conntrack entry's mark to the service tag. All subsequent packets of that flow automatically receive the correct DSCP mark without re-running the classifier.

7.3.1. Critical Kernel Module Dependency

Per-flow CONNMARK updates via `nfct_query(NFCT_Q_UPDATE)` require `kmod-nf-conntrack-netlink`, the kernel module that registers the conntrack netlink subsystem. This module is not installed by default on OpenWrt 24.10. The daemon detects this condition at runtime and logs a warning if CONNMARK updates fail with `EINVAL`. Installing the package (`opkg install kmod-nf-conntrack-netlink`) resolves the issue. This dependency is not documented in any OpenWrt QoS guide identified during this research.

7.4. Ingress Shaping via IFB

Download-side bufferbloat is addressed using an Intermediate Functional Block (IFB) device. At startup, MycoFlow:

1. Creates and brings up `ifb0`.
2. Attaches a `tc mirred` ingress redirect from the WAN interface to `ifb0`.
3. Installs a CAKE qdisc on `ifb0` with the same `diffserv4` mode and a configurable ingress rate.

Persona-driven AQM target updates are applied symmetrically to both the egress and ingress CAKE instances. The ingress bandwidth cap tracks egress adaptations: whenever the egress cap is adjusted by δ kbit/s, the ingress cap is adjusted by the same δ to maintain proportionality.

7.5. eBPF Telemetry

A BPF program (`mycflow.bpf.c`) is compiled for the TC hook using `clang -target bpf -g` with BTF generation. The program samples passive TCP RTT from the TCP timestamp option (TSval/TSsec pair) and records the elapsed time, maintaining a smoothed per-flow RTT in a `BPF_MAP_TYPE_HASH` map keyed by 5-tuple.

The eBPF integration is optional: when `libbpf` is unavailable or loading fails (e.g., kernel BTF not available), the daemon falls back gracefully to netlink-only statistics. The LuCI dashboard renders `rtt_ms=0` for all flows and suppresses the demotion indicator when BPF is inactive.

7.6. Configuration Hierarchy

Parameters are resolved in priority order: OpenWrt UCI (`/etc/config/mycflow`) → environment variables → compiled defaults. Key defaults are summarized in Table 7.1.

Table 7.1.: Key Configuration Parameters and Compiled Defaults

Parameter	Default	Description
<code>sample_hz</code>	1.0	Sense cycle frequency (Hz)
<code>ewma_alpha</code>	0.30	EWMA smoothing factor α
<code>rtt_margin</code>	0.30	Congestion margin γ
<code>baseline_decay</code>	0.01	Baseline drift rate β
<code>baseline_interval</code>	60	Cycles between baseline updates
<code>bw_step</code>	2000	BW step Δ (kbit/s)
<code>action_cooldown</code>	3.0	Min. action interval (s)
<code>baseline_samples</code>	5	Startup calibration samples
<code>per_device</code>	1	Enable per-device DSCP marking
<code>flow_aware</code>	1	Enable v3 per-flow classifier + CONNMARK
<code>rtt_bpf_obj</code>	—	Path to <code>mycflow_rtt.bpf.o</code>
<code>ingress_enabled</code>	0	Enable IFB ingress shaping

All parameters can be changed at runtime via `uci set` followed by `kill -HUP $(pidof mycflowd)` (SIGHUP triggers config reload and baseline recalibration without daemon restart).

7.7. Flash Safety

The daemon is designed to write zero bytes to NAND flash during normal operation—a critical requirement for routers with limited flash endurance. The target Xiaomi AX3000T has 128 MB NAND with a write endurance of approximately 10^5 program/erase cycles per block.

All state is RAM-resident:

- Log output → stdout → `procd` → logd ring buffer (RAM, typically 64 KB).

- State snapshot `/tmp/myco_state.json` $\rightarrow$ `tmpfs` (RAM disk, automatically cleared on reboot).
- Persona history and baseline values: in-process RAM variables; intentionally reset on reboot to ensure fresh baseline calibration.

7.8. Test Coverage

Thirteen CTest suites cover all modules across the core loop, the per-device persona layer, and the v3 flow-aware layer. All 13/13 suites pass (approximately 190 assertions total):

- *Core*: `test_ewma` (EWMA convergence with $\alpha \in \{0.1, 0.3, 0.9\}$); `test_act` (IFB setup/tear-down, interface-name injection guard, DSCP rule lifecycle); `test_control` (congestion, clear, outlier, feedback-ring); `test_config` (UCI parsing, env overrides, boundary clamping).
- *Persona layer*: `test_persona` (6-class decision tree, majority vote, edge cases); `test_device` (per-device aggregation, stale eviction, DSCP rule updates).
- *Flow-aware v3*: `test_service` (12-class taxonomy + demote ladder); `test_hint` (destination-port hint table); `test_dns` (passive DNS snoop cache, TTL expiry); `test_profile` (behavioral fingerprint scoring); `test_classifier` (3-signal voter, stability gate, mark commit); `test_mangle` (nftables DSCP-restore rule generation); `test_rtt` (BPF map parsing, demote/re-promote hysteresis).

8. Evaluation

This chapter reports the results of four controlled experiments designed to validate MycoFlow’s classification accuracy, QoS performance gains, and resource efficiency.

8.1. Experimental Setup

All QEMU-based experiments run inside a Docker container (privileged mode) hosting a QEMU 6.x virtual machine running OpenWrt 23.05.5 x86-64 (Linux 5.15, `sch_cake`, `iptables-nft`). The WAN link is emulated at 20 Mbit/s using `tc tbf`. Network namespaces represent distinct client hosts connected via an OpenWrt NAT bridge. `iperf3` servers listen on ports 5201–5210 in a dedicated server namespace. Latency is measured by 1 Hz ICMP echo requests from the gaming namespace to the server; CAKE tin statistics are collected via `tc -s qdisc show`. Each scenario runs for 30 seconds with a 3-second settling period before measurement begins.

The in-vivo hardware experiment (Phase 4) runs on the author’s live home router: Xiaomi AX3000T (MediaTek MT7981B, 2×Cortex-A53 @ 1.3 GHz, 256 MB LPDDR4, 128 MB NAND; OpenWrt 24.10.2, Linux 6.6).

8.2. Phase 1: Two-Persona Validation

8.2.1. Setup

Two client namespaces simulate a gamer (192.168.1.10) sending UDP at 300 kbps with 91-byte packets, and a bulk client (192.168.1.20) saturating the link with a single TCP stream. Three scenarios are compared: FIFO (no QoS), CAKE diffserv4 without MycoFlow, and CAKE + MycoFlow with two-persona classification.

8.2.2. Gaming Latency Results

Table 8.1 reports gaming latency under each scenario. FIFO induces 693 ms of queuing delay under link saturation with 12.5% packet loss—confirming the severity of unbuffered bufferbloat. CAKE eliminates the queuing delay entirely, reducing the loaded latency increase to +0.2 ms with zero packet loss, demonstrating the fundamental effectiveness of AQM. MycoFlow further reduces the loaded latency increase to +0.1 ms.

Table 8.1.: Phase 1: Two-Persona Gaming Latency (20 Mbit/s WAN)

Scenario	Idle (ms)	Loaded (ms)	Δ (ms)	Loss (%)
FIFO (no QoS)	0.45	693.82	+693.4	12.5
CAKE only	0.43	0.63	+0.2	0.0
CAKE + MycoFlow	0.58	0.71	+0.1	0.0

8.2.3. CAKE Tin Statistics

The key result of Phase 1 is the tin-level delay separation (Table 8.2). Without MycoFlow, all traffic lands in Best Effort; peak in-tin delay reaches 6.2 ms. With MycoFlow, the gamer is steered to the Voice tin (CS4) and the bulk client to the Bulk tin (CS1), resulting in a **154 $\times$ difference** in average in-tin delay: Voice at 225 μ s versus Bulk at 34.7 ms.

Table 8.2.: Phase 1 CAKE Tin Delay: CAKE-only vs. MycoFlow

Tin	Avg Delay		Packets	
	CAKE	+MycoFlow	CAKE	+MycoFlow
Voice	—	225 μs	0	6 015
Best Effort	4.2 ms	4.2 ms	111 910	68 185
Bulk	—	34.7 ms	0	136 694

This result isolates the core contribution of per-device classification: even without flow-level awareness, correctly marking the gamer as GAMING produces a 154 $\times$ tin delay advantage over an unmanaged baseline.

8.3. Phase 2: Six-Persona Mixed Workload

8.3.1. Setup

Six client namespaces simulate a realistic home network (Table 8.3). Five scenarios (S1–S5) cover FIFO, CAKE-only, and MycoFlow configurations in both 2-client and 5-client variants.

Table 8.3.: Phase 2 Client Workload Profiles

Namespace	Protocol	Rate	Pkt Size
gamer	UDP (iperf3)	300 kbps	$\sim$ 120 B
voip	UDP (iperf3)	64 kbps	64 B
video	UDP bidirectional	2 Mbps	800 B
stream	TCP reverse	15 Mbps	1400 B
torrent	TCP (30 parallel)	WAN cap	1400 B

8.3.2. Latency Results

Table 8.4 summarizes the five-scenario results. The $818\times$ improvement of CAKE over FIFO (from 558 ms to 0.68 ms) confirms the baseline necessity of AQM. Under the 5-client workload (S4 vs. S5), MycoFlow reduces gaming latency by **38%** (1.43 ms $\rightarrow$ 1.10 ms).

Table 8.4.: Phase 2 Gaming Latency: Five Scenarios

#	Scenario	Idle (ms)	Loaded (ms)	Δ (ms)
S1	FIFO	0.52	558.03	+557.5
S2	CAKE, 2-client	0.59	0.68	+0.1
S3	CAKE+MF, 2-pers.	0.63	0.69	+0.1
S4	CAKE, 5-client	0.59	1.43	+0.8
S5	CAKE+MF, 6-pers.	0.56	1.10	+0.5

8.3.3. CAKE Tin Distribution

Table 8.5 compares tin occupancy between S4 (CAKE only) and S5 (MycoFlow). Without MycoFlow, all 191,188 packets share Best Effort with 42,557 drops and a 9.5 ms average in-tin delay. With MycoFlow, traffic separates across three tins: 95,199 torrent packets move to Bulk (29.9 ms avg delay), 37,237 packets enter Video (252 μ s avg delay), and 59,751 remaining Best Effort packets experience only 304 μ s average delay. The Voice tin shows 2 μ s average delay for gaming and VoIP.

Table 8.5.: Phase 2 CAKE Tin Statistics: S4 vs. S5

Tin	Avg Delay		Packets	
	S4	S5	S4	S5
Voice	—	2 μ s	1	2
Video	—	252 μ s	0	37 237
Best Effort	9.5 ms	304 μ s	191 188	59 751
Bulk	—	29.9 ms	0	95 199
Drops	42 557	34 528	—	—

The 38% improvement operates through two channels: direct isolation (95,199 torrent packets moved to Bulk) and Best Effort de-congestion (fewer competing flows allow CAKE’s per-flow fair queuing to more effectively isolate gaming ICMP packets, reducing Best Effort delay from 9.5 ms to 304 μ s).

8.4. Phase 3: Single-IP Multi-App Flow Separation

8.4.1. Motivation

Phase 2 edge cases reveal a structural limitation of per-device classification: when a single host runs multiple applications simultaneously, one persona must be chosen for all traffic. Phase 3 tests whether the v3 flow-aware classifier resolves this limitation.

8.4.2. Setup

All four applications run from the same gamer network namespace (single src IP 192.168.1.10): a UDP stream to port 27015 (`GAME_RT`), a UDP stream to port 3478 (`VOIP_CALL`), a TCP stream to port 9000 (`BULK_DL`), and a 20-way parallel TCP stream to port 6881 (`TORRENT`). Two modes are compared over $N = 15$ independent runs: Mode A (`flow_aware=0`, per-device only) and Mode B (`flow_aware=1`, per-flow v3).

8.4.3. Results

Table 8.6 reports the $N=15$ multi-run results. In Mode A, the 20-way torrent stream dominates the per-device persona vote, assigning CS1 (Bulk tin) to all four applications; only 3 Voice-tin packets per run are observed. In Mode B, the `GAME_RT` and `VOIP_CALL` streams receive EF and fill the Voice tin with $14,193 \pm 21$ packets per run ($p < 10^{-40}$, Welch t -test). ICMP RTT delta is significantly lower in Mode B (0.216 vs. 0.292 ms, $p = 0.020$).

Table 8.6.: Phase 3: Per-Device vs. Per-Flow Under Single-IP Multi-App Load ($N=15$ runs)

Mode	Voice-tin (pkts)	Δ RTT (ms)	95% CI (ms)	p
Per-device (v2)	3 ± 1	0.292	[0.241, 0.343]	—
Per-flow (v3)	$14,193 \pm 21$	0.216	[0.173, 0.259]	0.020

8.5. Phase 4: DNS Signal Ablation and In-Vivo Classifier Accuracy

8.5.1. Motivation

Phases 1–3 measure QoS outcome on synthetic QEMU workloads. Phase 4 directly quantifies classifier accuracy on real-world traffic using the public Universidad del Cauca (Unicauca) dataset [15] (2.4M labeled flows captured April–June 2019 at a university gateway).

8.5.2. Offline Temporal Validation

The dataset is split at 2019-06-01: calibration (Apr–May) and a held-out set (June). No development decisions were made after inspecting the June partition. Table 8.7 shows that the

calibration-to-holdout accuracy gap is at most -0.44 pt, confirming that MycoFlow’s heuristic thresholds generalize across the calibration boundary without temporal overfitting.

Table 8.7.: Temporal Generalization Gap on the Unicauca Dataset

Signal Set	Apr–May	June (held-out)	Δ
Port + behavior	42.95%	42.51%	-0.44
Port + behavior + DNS	47.43%	47.04%	-0.39

8.5.3. In-Vivo Validation

For each of 11 service classes, $N = 150$ flows are drawn via stratified random sampling (seed=42) from Unicauca and replayed against the live Xiaomi AX3000T in two ablation modes:

- **Mode A (IP-only):** Traffic directed to fixed public IPs without prior DNS queries; $h_{dns} = 0$ for all flows.
- **Mode B (hostname):** A DNS A-record query for the flow’s canonical hostname is issued before traffic begins.

Table 8.8 reports the results across five runs on two testbeds (WSL2 laptop and bare-metal Ubuntu 24.04).

Table 8.8.: In-Vivo Classifier Accuracy ($N = 1,650$ stratified-random flows, Wilson 95% CI for the best run)

Run	Svc-acc	Tin-acc	Tin-match
WSL Mode A	20.00%	30.42%	49.46%
WSL Mode B	32.48%	45.39%	64.42%
Ubuntu Mode A	20.24%	30.61%	49.26%
Ubuntu Mode B (80 dom.)	33.39%	48.61%	66.97%
Ubuntu Mode B (585 dom.)	37.88%	52.42%	70.41%
	[35.6, 40.3]	[50.0, 54.8]	[67.6, 73.1]
Mode A vs. B invariance (WSL vs. Ubuntu): $ \Delta < 0.5$ pt			
DNS lift: +17.6 pt svc-acc, +21.2 pt tin-match (585-domain)			
McNemar ($p < 10^{-63}$): $b=291$, $c=0$			

The tin-matched accuracy with the full 585-domain DNS catalog reaches **70.41%** [67.6%, 73.1%] on the bare-metal Ubuntu testbed. McNemar’s paired test rejects the null of equal classifiers at $p = 1.8 \times 10^{-63}$: 291 flows are correctly assigned by Mode B but missed by Mode A, and *zero* flows are degraded by DNS evidence—DNS is strictly additive.

8.5.4. AF_PACKET Requirement

A naive implementation using `AF_INET SOCK_RAW` captures only packets destined for the router’s own IP, missing transit DNS from clients with hard-coded public resolvers (1.1.1.1, 8.8.8.8). In a pre-fix run, Mode B accuracy (19.1%) was statistically indistinguishable from Mode A (18.6%). Replacing the socket with `AF_PACKET SOCK_DGRAM ETH_P_IP` captures all IPv4 frames on the LAN-facing interface; after the fix, Mode B reached 32.5% vs. Mode A’s 20.0%—a +12.5 pt lift ($p < 10^{-44}$).

8.6. Resource Overhead

Table 8.9 summarizes the resource footprint measured on the live Xiaomi AX3000T with the full v3 stack active ($n=523$ clean one-second samples during normal residential traffic).

Table 8.9.: Resource Overhead (AX3000T, /proc-sampled, $n=523$ samples, full v3 stack)

Metric	p50	p95	max
System CPU (%)	17	43	61
mycoflowd CPU (%)	6	8	12
Conntrack entries	357	442	503
CT marks active	56	79	117
mycoflowd RSS	1,152 KB (constant)		
Flash (mtd)	0 writes		
Crashes	0 / 523		

Key observations: (1) RSS held constant at 1,152 KB across all 523 samples—the daemon is allocation-static after startup; (2) median CPU is 6% on a single 1.3 GHz A53 core; (3) median 56 conntrack entries carry per-flow CT marks, confirming the CONNMARK pipeline is operational; (4) zero flash writes and zero crashes across the measurement window.

9. Discussion

This chapter discusses the interpretation of the experimental results, the limitations of the current system, and the implications for future deployment and research.

9.1. Why 38% and Not More?

The primary limiting factor in the six-persona scenario (S5) is the gamer’s misclassification as UNKNOWN. In the two-client scenario (S3), the gamer correctly enters the Voice tin at 2 μ s average delay—a 5000 $\times$ improvement over its unmanaged Best Effort delay. The misclassification occurs because the `iperf3` benchmark framework generates additional control flows that inflate the gamer’s flow count above the GAMING threshold ($F < 8$).

In real-world deployment, gaming clients (Steam, consoles) generate 1–4 persistent UDP flows at low packet rates with small packets—an unambiguous GAMING signal. The benchmark edge case is an artifact of the test harness, not an inherent limitation of the classifier. Routing benchmark control flows separately from data flows (e.g., via a dedicated management VLAN) would eliminate the false UNKNOWN classification and is expected to push the S5 improvement well beyond 38%.

9.2. Classification Accuracy Discussion

9.2.1. Per-Device Heuristic Design

The six-persona heuristic was designed for real household device behavior, not for synthetic benchmarks. The decision tree parameters (flow count thresholds, packet size ranges, throughput bounds) were derived from analysis of typical device traffic patterns:

- Real gaming clients: 1–4 persistent UDP flows, small packets (60–120 B), low throughput (50–500 kbps).
- Real streaming clients: 1–3 long-lived TCP connections, large packets (1400 B), high sustained throughput (5–25 Mbps).
- Real VoIP clients: 1–2 UDP flows, very small packets (40–80 B), very low throughput (<100 kbps).

The benchmark edge cases (inflated flow counts, artificially low traffic volumes) do not represent these patterns.

9.2.2. In-Vivo Classification Ceiling

The in-vivo classifier accuracy of 70.4% tin-matched accuracy represents the current ceiling of the three-signal voter on real-world traffic. The primary bottleneck is the DNS catalog completeness: with only 80 domains, tin-matched accuracy is 67.0%; with 585 domains, it rises to 70.4%. Expanding the catalog further is expected to yield additional gains.

The remaining 29.6% of flows that are incorrectly classified fall into three categories:

1. **Encrypted traffic without DNS pre-queries:** Applications that use QUIC or WireGuard tunnels without resolvable hostnames cannot be classified by port hints or DNS.
2. **Non-standard ports:** Applications that use random or dynamic ports not covered by the hint table.
3. **Behavioral ambiguity:** Flows with packet size and rate characteristics that match multiple service classes (e.g., a slow TCP download that resembles a VoIP call).

9.3. Comparison with Two-Persona Results

The two-persona Phase 1 result ($154\times$ tin delay differential) is a stronger isolation proof than the six-persona 38% improvement, because in Phase 1 the gamer is correctly classified. Together, the two experiments demonstrate complementary aspects of MycoFlow’s effectiveness:

1. **Ideal case (correct classification):** MycoFlow delivers dramatic isolation ($154\times$ tin delay difference), confirming that DSCP-based steering is highly effective when classification is accurate.
2. **Adversarial case (misclassification):** Even with one device misclassified as UNKNOWN, the system still reduces gaming latency meaningfully (38%) by isolating high-volume background traffic.

9.4. Adaptive Baseline vs. Static Threshold

The sliding EWMA baseline ensures the controller does not become permanently anchored to favorable boot-time conditions. Consider a router rebooted at 3 AM when ambient RTT is 10 ms: a 30% margin yields $\theta_r = 13$ ms. Six hours later during peak hours, ambient RTT may be 25 ms; without baseline adaptation, the controller would fire on every cycle. With $\beta = 0.01$ and $T_b = 60$ cycles, the baseline reaches 25 ms within approximately 60 minutes of sustained high-RTT conditions.

9.5. Parameter Sensitivity

MycoFlow’s heuristic thresholds ($\alpha = 0.30$, $\gamma = 0.30$, $k = 3/m = 5$, $\Delta = 2,000$ kbit/s) were set empirically from a pilot run and have not been varied in a controlled experiment. The parameters with the strongest expected impact are:

- **Majority-vote window (k/m):** A lower ratio increases responsiveness at the cost of more oscillation; a higher ratio increases stability at the cost of slower adaptation.
- **EWMA factor (α):** A higher α accelerates convergence but amplifies noise; a lower α is more stable but slower to react.

A systematic grid search over these parameters is an explicit item in the future work roadmap.

9.6. Privacy and Security

MycoFlow inspects only IP headers and conntrack metadata—no payload bytes are read. The passive DNS snooping captures only DNS response packets (UDP/53 and UDP/853), not the content of encrypted flows. The behavioral fingerprint records only aggregate statistics (mean packet size, packets per second, inter-arrival jitter) that are observable from header information.

The eBPF program is loaded into the kernel with standard capability separation; the userspace daemon runs without root after initial setup (capability dropping is planned for a future release). The ubus interface is protected by OpenWrt’s native ACL mechanism; all bandwidth and DSCP commands are rate-limited.

9.7. Threats to Validity

1. **QEMU overhead:** QEMU introduces CPU overhead not present on bare metal; the FIFO and CAKE-only baselines share this overhead, making relative comparisons valid but absolute latency values slightly inflated.
2. **Short scenario duration:** The 30-second scenario duration may exhibit higher variance than longer real-world runs; Phase 3 ($N=15$ runs) shows $\sigma \approx 0.09$ ms on RTT delta, consistent with the estimated ± 0.2 ms bound.
3. **Benchmark control traffic:** The test harness injects control traffic that inflates flow counts; this is a measurement artifact, not a production scenario.
4. **Replayed flow fidelity:** Phase 4 replayed flows preserve per-flow median packet size and rate but not bursty arrival patterns; the behavioral fingerprint signal ($w = 0.1$) is consequently weaker than in live deployment.

5. **Ground truth quality:** Unicauca uses nDPI-based ground truth, which has its own finite accuracy; a flow labeled `voip_call` may in fact be DNS overhead from a VoIP application.
6. **Hardware generalization:** Generalization to other consumer SoCs (Qualcomm, Broadcom) is plausible given OpenWrt's portability but not directly measured.

9.8. Limitations

- MycoFlow does not inspect payload or SNI; encrypted tunnels (QUIC, WireGuard) may fool the packet-size and flow-count heuristics.
- The 1 Hz control rate introduces a reaction latency of up to 1 s between the onset of congestion and the first actuation.
- The `voip_call` service class achieves only 1.3% recall in both ablation modes, a notable weakness caused by the Unicauca dataset conflating multiple application types under the same label.
- The per-flow RTT demote/re-promote ladder is implemented and active on the live router, but a controlled evaluation of its isolated latency impact is deferred to future work.

10. Conclusion

This thesis has presented MycoFlow, a bio-inspired reflexive QoS daemon for OpenWrt routers that addresses the two fundamental limitations of consumer-grade home network management: the inability to automatically classify traffic without DPI or machine learning, and the inability to adapt the WAN bandwidth cap to real-time congestion signals.

10.1. Summary of Contributions

The primary contributions of this work are:

1. **A two-level traffic classification architecture** combining a six-class per-device persona engine (operating on aggregate device-level behavioral metrics) and a twelve-class per-flow service taxonomy (operating on individual conntrack 5-tuples), both driven by heuristic decision trees without DPI or ML.
2. **A three-signal weighted voter** that combines passive DNS snooping, destination-port hints, and behavioral fingerprints with learned weights $(w_{\text{dns}}, w_{\text{port}}, w_{\text{behav}}) = (0.6, 0.3, 0.1)$, enabling robust classification of encrypted traffic where port hints alone fail.
3. **A practical AF_PACKET fix for in-router DNS sniffing**, demonstrating that AF_INET SOCK_RAW silently misses transit DNS from clients with hard-coded public resolvers—a trap that reduces DNS classification lift by 12.5 percentage points with no error indication.
4. **A feedback-driven hysteresis controller** with sliding EWMA baseline, action feedback ring, and per-flow RTT demote/re-promote ladder, enabling closed-loop bandwidth adaptation without oscillation.
5. **A quantitative in-vivo DNS ablation study** ($N = 1,650$ flows, Universidad del Cauca dataset) with Wilson 95% confidence intervals and McNemar testing, providing rigorous evidence that passive transit-DNS is the most informative classification signal, lifting tin-matched accuracy from 30% to 70.4% ($p < 10^{-63}$, $c = 0$ flows degraded).

10.2. Experimental Findings

Four controlled experiments demonstrate the system’s effectiveness:

- **Phase 1 (two-persona):** A $154\times$ in-tin delay advantage for gaming over bulk ($225\ \mu\text{s}$ vs. $34.7\ \text{ms}$) when per-device classification is correct, confirming that DSCP-based tin steering delivers dramatic isolation.
- **Phase 2 (six-persona, 5 clients):** A 38% reduction in gaming latency ($1.43\ \text{ms} \rightarrow 1.10\ \text{ms}$) under a mixed-persona workload, achieved by isolating 95,199 torrent packets into the Bulk tin and reducing Best Effort congestion from $9.5\ \text{ms}$ to $304\ \mu\text{s}$ average delay.
- **Phase 3 (single-IP multi-app):** Correct per-flow service separation when a single source IP simultaneously runs game, VoIP, bulk, and torrent traffic, filling the CAKE Voice tin with $14,193 \pm 21$ packets per run vs. 3 ± 1 without per-flow classification ($p < 10^{-40}$).
- **Phase 4 (in-vivo DNS ablation):** Passive transit-DNS lifts tin-matched accuracy from 30% to 70.4%, with zero flows degraded by DNS evidence and no temporal overfitting ($\Delta_{\text{cal} \rightarrow \text{holdout}} < 0.5\ \text{pt}$).

The daemon runs at constant 1,152 KB RSS (including statically linked CONNMARK libraries) with median 6% CPU on a single 1.3 GHz Cortex-A53 core and writes zero bytes to NAND flash, making it deployable on any OpenWrt-compatible consumer router without hardware modifications.

10.3. Bio-Inspired Design Validation

The bio-inspired design philosophy—local sensing, adaptive redistribution, hysteresis-guided stability, and continuous baseline recalibration—proved well-suited to the embedded QoS domain. Each mycelial principle maps cleanly to an engineering mechanism: hyphal sensing to multi-source metric collection, adaptive transport to DSCP-based tin steering, hysteresis to majority-vote stability, and load-triggered remodeling to sliding baseline adaptation.

The system achieves its objectives without requiring external infrastructure: no cloud service, SDN controller, model training pipeline, or payload inspection capability is needed. This design simplicity is both a practical advantage for consumer deployment and a validation of the mycelial analogy—complex adaptive behavior emerging from simple local rules.

10.4. Future Directions

Several extensions are planned for future work:

- **Cooperative “mycelium maps”:** Neighboring routers could exchange aggregated congestion signals over a lightweight IPC protocol, enabling coordinated bandwidth adaptation without a central controller.
- **Sub-second eBPF-driven sensing:** Replacing the 1 Hz ICMP probe with BPF_MAP_TYPE_RINGBUF-based event streaming would reduce the congestion detection reaction time from 1 s to under 100 ms.

- **Optional TinyML persona prediction:** A strict 1 MB model budget for a gradient-boosted tree trained on conntrack features could handle encrypted tunnel classification cases that the heuristic decision tree cannot resolve.
- **STUN/RTP application-layer fingerprinting:** Adding recognition of STUN session establishment and RTP header patterns would resolve the voip_call recall limitation identified in Phase 4.
- **Parameter optimization:** A systematic grid search over $(\alpha, \gamma, k/m, \Delta)$ under controlled workloads would provide principled guidance for parameter selection across different deployment environments.
- **Enterprise deployment:** SDN integration via OpenWrt's ubus IPC would allow coordinated DSCP policy enforcement across managed enterprise-grade deployments.

10.5. Acknowledgements

The author thanks the OpenWrt community for CAKE and the embedded Linux QoS infrastructure; the Linux kernel team for eBPF, conntrack, and netlink subsystems; and the maintainers of QEMU and Docker for enabling reproducible embedded-systems research. Special thanks to Dr. Cihat Çetinkaya for guidance throughout this project.

A. Additional Materials

Bu bölümde tezde yer almayan ek materyaller sunulmaktadır.

A.1. Configuration Parameters Reference

Table A.1 provides a complete listing of all MycoFlow UCI configuration parameters, their types, default values, and acceptable ranges.

Table A.1.: Complete MycoFlow UCI Configuration Reference

Parameter	Type	Default	Description
sample_hz	float	1.0	Sense cycle frequency; range [0.1, 10.0] Hz
ewma_alpha	float	0.30	EWMA smoothing factor; range (0, 1)
rtt_margin	float	0.30	Congestion threshold margin γ ; range (0, 1)
baseline_decay	float	0.01	Baseline drift rate β ; range (0, 0.1)
baseline_interval	int	60	Cycles between baseline updates
bw_step	int	2000	Bandwidth adjustment step (kbit/s)
bw_min	int	1000	Minimum WAN cap (kbit/s)
bw_max	int	1000000	Maximum WAN cap (kbit/s)
action_cooldown	float	3.0	Minimum action interval (s)
baseline_samples	int	5	Startup calibration sample count
per_device	bool	1	Enable per-device DSCP marking
flow_aware	bool	1	Enable v3 per-flow classifier + CONNMARK
ingress_enabled	bool	0	Enable IFB ingress shaping
wan_interface	string	eth0	WAN interface name
probe_host	string	1.1.1.1	ICMP probe target host
rtt_bpf_obj	string	—	Path to mycoflow_rtt.bpf.o

A.2. Unicauca Dataset Service Class Mapping

Table A.2 shows the mapping between Unicauca dataset application labels and MycoFlow service classes used in the Phase 4 evaluation.

Table A.2.: Unicauca Application Label to MycoFlow Service Class Mapping

Unicauca Label	MycoFlow Service Class
Game (UDP)	game_rt
VoIP	voip_call
Video Call	video_conf
Live Streaming	video_live
Video on Demand	video_vod
Web Browsing	web_interactive
File Transfer	file_sync
P2P (BitTorrent)	torrent
FTP / Bulk	bulk_dl
Game Launcher	game_launcher
DNS / System	system

A.3. CAKE Tin Target Delay Summary

CAKE's internal AQM target delay is computed from the `rtt` parameter as $\text{target} \approx \text{rtt}/20$. Table A.3 shows the AQM targets applied per persona:

Table A.3.: Persona-Driven CAKE AQM Target Delays

Persona Group	CAKE rtt param	AQM target
VOIP, GAMING	50 ms	2.5 ms
VIDEO	75 ms	3.75 ms
STREAMING	150 ms	7.5 ms
BULK, TORRENT	200 ms	10 ms
UNKNOWN	100 ms	5 ms (CAKE default)

A.4. Build and Installation Instructions

A.4.1. Building from Source

```
# Install OpenWrt SDK for aarch64-openwrt-linux-musl
# Set SDK_PATH to the SDK root directory
```

```
mkdir build && cd build
```

```
cmake .. -DCMAKE_TOOLCHAIN_FILE=${SDK_PATH}/toolchain.cmake \  
        -DCMAKE_BUILD_TYPE=Release  
make -j$(nproc)  
ctest --output-on-failure # Run all 13 test suites
```

A.4.2. OpenWrt Installation

```
# Copy binary to router  
scp mycoflowd root@192.168.1.1:/usr/sbin/  
  
# Install required kernel module (OpenWrt 24.10+)  
opkg install kmod-nf-contrack-netlink  
  
# Configure via UCI  
uci set mycoflow.main.wan_interface=eth1  
uci set mycoflow.main.flow_aware=1  
uci commit mycoflow  
  
# Start daemon  
/etc/init.d/mycoflow start
```

List of Figures

- 5.1. MycoFlow reflexive control loop. Each phase executes once per second. 16
- 5.2. Per-flow data path from packet arrival to CAKE tin assignment. 18

List of Tables

3.1. Comparison of Related QoS Approaches. Emb.=runs fully on CPE; Auto=automatic classification; No ML=no ML model required; No DPI=no payload inspection; Adapt=runtime adaptation.	10
4.1. Mycelial Principles Mapped to MycoFlow Mechanisms	12
5.1. MycoFlow Source Modules and Responsibilities	17
6.1. Persona Inference Decision Table (first matching rule wins)	20
6.2. Persona-to-DSCP-to-CAKE-Tin Mapping	21
6.3. Flow-Level Service Taxonomy	21
7.1. Key Configuration Parameters and Compiled Defaults	26
8.1. Phase 1: Two-Persona Gaming Latency (20 Mbit/s WAN)	29
8.2. Phase 1 CAKE Tin Delay: CAKE-only vs. MycoFlow	29
8.3. Phase 2 Client Workload Profiles	29
8.4. Phase 2 Gaming Latency: Five Scenarios	30
8.5. Phase 2 CAKE Tin Statistics: S4 vs. S5	30
8.6. Phase 3: Per-Device vs. Per-Flow Under Single-IP Multi-App Load ($N=15$ runs)	31
8.7. Temporal Generalization Gap on the Unicauca Dataset	32
8.8. In-Vivo Classifier Accuracy ($N = 1,650$ stratified-random flows, Wilson 95% CI for the best run)	32
8.9. Resource Overhead (AX3000T, /proc-sampled, $n=523$ samples, full v3 stack)	33
A.1. Complete MycoFlow UCI Configuration Reference	41
A.2. Unicauca Application Label to MycoFlow Service Class Mapping	42
A.3. Persona-Driven CAKE AQM Target Delays	42

Bibliography

- [1] Jim Gettys and Kathleen Nichols. Bufferbloat: Dark buffers in the Internet. *ACM Queue*, 9(11):40–54, 2011. doi: 10.1145/2063176.2071893.
- [2] Kathleen Nichols and Van Jacobson. Controlling queue delay. *ACM Queue*, 10(5), 2012.
- [3] Toke Høiland-Jørgensen et al. Piece of CAKE: A comprehensive queue management solution for home gateways. In *Proc. IEEE LANMAN*, pages 1–6, 2018.
- [4] Toke Høiland-Jørgensen, Jonathan Morton, Dave Taht, and Jim Gettys. The CAKE algorithm for network shaping and AQM. In *Proc. IETF Applied Networking Research Workshop*, 2018.
- [5] Kathleen Nichols, Steven Blake, Fred Baker, and David Black. Definition of the differentiated services field (DS field) in the IPv4 and IPv6 headers. Technical Report RFC 2474, IETF, 1998.
- [6] Rong Pan, Preethi Natarajan, Chiara Piglion, Mythili S. Prabhu, Vijay Subramanian, Fred Baker, and Bill VerSteeg. PIE: A lightweight control scheme to address the bufferbloat problem. In *Proc. IEEE HPSR*, pages 148–155, 2013.
- [7] Martino Trevisan, Marco Mellia, Romain Chabanne, and Pedro Casas. GGFAST: Graph-based generative framework for accurate session-type classification. In *Proc. ACM SIGCOMM*, pages 1–14, 2023. doi: 10.1145/3603269.3604840.
- [8] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Reza Shirali Hossein Zade, and Mohammad-sadegh Saberian. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing*, 24(3):1999–2012, 2020.
- [9] Luca Deri, Maurizio Martinasso, Roberto Breglia, and Marco Sabetta. nDPI: Open-source high-speed deep packet inspection. In *Proc. IEEE TRAC*, 2014.
- [10] Simone Magnani, Fulvio Rizzo, and Domenico Siracusa. A control plane enabling automated and fully adaptive network traffic monitoring with eBPF. *IEEE Access*, 10: 93622–93641, 2022. doi: 10.1109/ACCESS.2022.3202644.
- [11] Marco Dorigo, Vittorio Maniezzo, and Alberto Coloni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1):29–41, 1996.
- [12] Andrew Adamatzky et al. Logics in fungal mycelium networks. *Logica Universalis*, 16: 55–80, 2022.

- [13] Mark D. Fricker, Luke L. M. Heaton, Nick S. Jones, and Lynne Boddy. The mycelium as a network. *Microbiology Spectrum*, 5(3), 2017.
- [14] Yu Fukasawa, Morag Savoury, and Lynne Boddy. Foraging strategies of fungal mycelial networks. *Frontiers in Cell and Developmental Biology*, 2023.
- [15] Juan Sebastian Rojas. IP network traffic flows labeled with 75 apps. Universidad del Cauca, Popayon, Colombia, 2019. URL <https://www.kaggle.com/datasets/jsrojas/ip-network-traffic-flows-labeled-with-87-apps>.